



Lập trình Android

Tổng quan

➤ Android là gì?

- Hệ điều hành dựa trên nền tảng Linux được thiết kế cho các thiết bị di động thông minh và máy tính bảng.
- Được phát triển bởi Open Handset Alliance (Google và các công ty khác)
- Hệ điều hành mã nguồn mở.
- Có sẵn một số ứng dụng gốc hỗ trợ cho điện thoại, nhắn tin, email, quản lý danh bạ, lịch, giải trí, định vị, giải trí đa Phương tiện...
- Các nhà phát triển Java bên thứ ba có thể sử dụng Android API để mở rộng các chức năng của thiết bị
- Google cung cấp một thị trường điện tử trực tuyến cho các nhà phát triển bên thứ ba có thể bán ứng dụng tùy chỉnh của họ

Tổng quan



Music



Sports



Travel



Business



Social Media



News



Lifestyle



Weather



Reference



Utilities



Multimedia



Food & Drink



Books



Navigation



Finance

Tổng quan

➤ Lịch sử phát triển:

- Android, Inc., một công ty nhỏ được thành lập vào vào tháng 10/2003
- Tháng 7/2005, Google mua lại
- Tháng 11/2007, Liên minh thiết bị cầm tay mở rộng (bao gồm Google và các công ty khác) được thành lập và phiên bản beta đầu tiên (Android Software Development Kit – SDK, dựa trên nhân Linux 2.6) cũng ra mắt cùng ngày.
- Tháng 9/2008, phiên bản thương mại đầu tiên, Android 1.0 được phát hành.
- Trải qua nhiều lần cập nhật, đến ngày 27/06/2012, Google đã công bố phiên bản Android 4.1 Jelly Bean: đây là một bản cập nhật đáng kể với mục đích chính là cải thiện giao diện người dùng cả về chức năng lẫn hiệu suất.

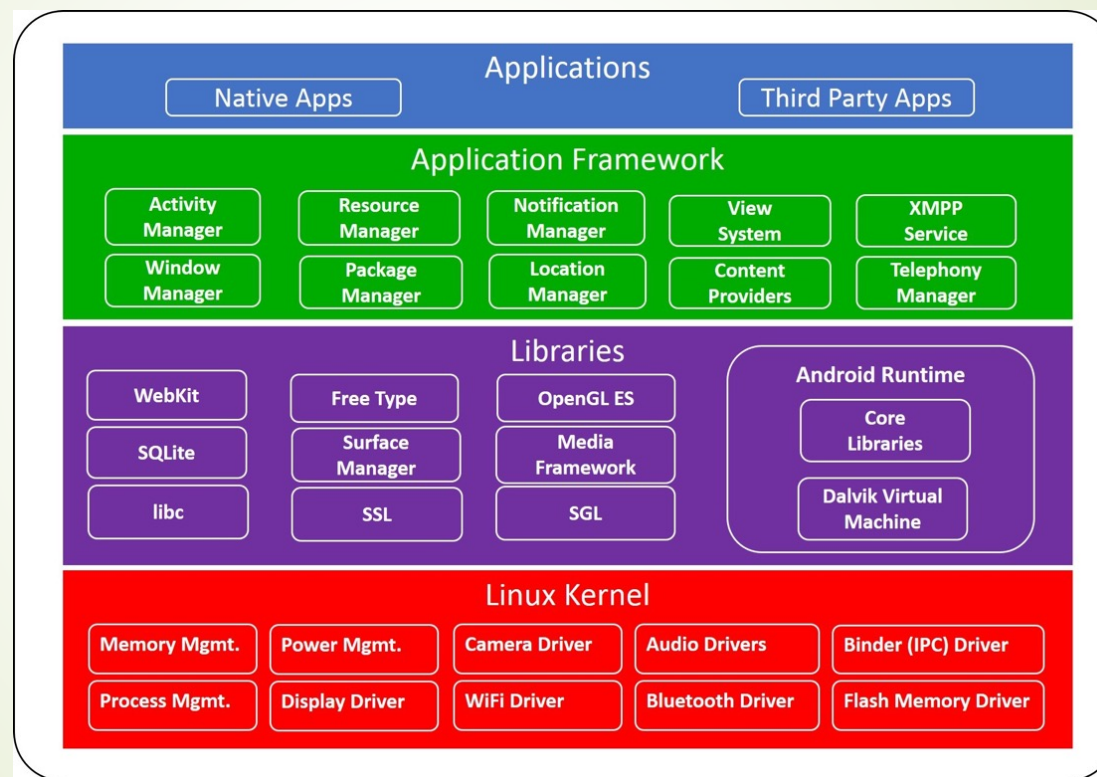


Tổng quan

➤ Tính năng

- **Application framework** cho phép tái sử dụng code và các thành phần
- Sử dụng **máy ảo Dalvik** tối ưu hóa hoạt động các thiết bị di động
- **Trình duyệt tích hợp** dựa trên công cụ WebKit mã nguồn mở
- **Đồ họa tối ưu hóa** được hỗ trợ bởi thư viện đồ họa 2D, 3D dựa trên OpenGL ES 1.0
- **SQLite** hỗ trợ lưu trữ dữ liệu cấu trúc
- **Hỗ trợ Media** với nhiều định dạng về âm thanh, hình ảnh, video (MPEG4, H.264, MP3, AAC, AMR, JPG, PNG, GIF)
- **Điện thoại GSM** (phụ thuộc vào phần cứng)
- **Bluetooth, EDGE, 3G, 4G, và Wi-Fi** (phụ thuộc vào phần cứng)
- **Camera, GPS, la bàn, và gia tốc kế** (phụ thuộc vào phần cứng)
- **Môi trường phát triển phong phú** bao gồm trình giả lập thiết bị, gỡ lỗi, quản lý bộ nhớ, sự thực thi...kết hợp với plugin cho Eclipse IDE

Kiến trúc



Kiến trúc

■ LINUX KERNEL

- Tầng dưới cùng với nhiều patch
- Điều này cung cấp một mức độ trừu tượng giữa phần cứng thiết bị và nó chứa tất cả các trình điều khiển phần cứng thiết yếu như máy ảnh, bàn phím, màn hình, v.v.
- Ngoài ra, kernel xử lý tất cả những thứ mà Linux làm rất tốt, chẳng hạn như mạng và một loạt các trình điều khiển thiết bị, giúp giảm bớt khó khăn khi giao tiếp với phần cứng ngoại vi.



Kiến trúc

LIBRARIES

Tầng tiếp theo của Kernel là **bộ thư viện** mã nguồn mở như:

- Công cụ trình duyệt web Webkit
- Thư viện chuẩn C libc
- SQLite
- Thư viện hỗ trợ thu phát âm thanh và video
- Thư viện SSL hỗ trợ cho bảo mật Internet
- ...

➤ Ví dụ: android.app, android.content, android.database, android.opengl, android.os, android.text, android.view, android.widget, android.webkit, ...



Kiến trúc

➤ ANDROID RUNTIME

- Thành phần chính của Android Runtime là Dalvik Virtual Machine (Dalvik VM)
- **Dalvik VM là một loại máy ảo Java** được thiết kế đặc biệt và tối ưu hóa cho Android
- Dalvik VM sử dụng các tính năng cốt lõi của Linux như quản lý bộ nhớ và đa luồng, vốn có trong ngôn ngữ Java. Dalvik VM cho phép mọi ứng dụng Android chạy trong tiến trình riêng của nó, với phiên bản máy ảo Dalvik của riêng nó.
- Android Runtime cũng cung cấp một bộ thư viện cốt lõi cho phép các nhà phát triển ứng dụng Android viết các ứng dụng Android bằng ngôn ngữ lập trình Java tiêu chuẩn.



Kiến trúc

■ APPLICATION FRAMEWORK

- Tầng Application Framework cung cấp nhiều dịch vụ cấp cao hơn cho các ứng dụng dưới dạng các lớp Java.
- Các nhà phát triển ứng dụng được phép sử dụng các dịch vụ này trong các ứng dụng của họ.



Kiến trúc

➤ APPLICATION FRAMEWORK

- **Activity Manager:** kiểm soát tất cả các khía cạnh của vòng đời ứng dụng và ngăn xếp hoạt động.
- **Content Providers:** cho phép ứng dụng xuất bản và chia sẻ dữ liệu với các ứng dụng khác.
- **Resource Manager:** cung cấp quyền truy cập vào các tài nguyên được nhúng không mã như chuỗi, cài đặt màu và bố cục giao diện người dùng.



Kiến trúc

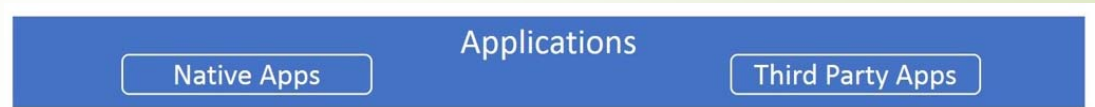
➤ APPLICATION FRAMEWORK

- **Notifications Manager:** cho phép ứng dụng hiển thị cảnh báo và thông báo cho người dùng.
- **View System:** tập hợp các dạng view được sử dụng để tạo giao diện người dùng ứng dụng.



Kiến trúc

➤ APPLICATIONS



- Ứng dụng mà bạn viết sẽ được cài đặt ở đây
- Bạn sẽ tìm thấy các ứng dụng của Android ở lớp trên cùng này
- Ví dụ: Contacts, Browser, Games,...





Ngôn ngữ lập trình

- ▀ Sử dụng ngôn ngữ lập trình Java
- 



Môi trường phát triển ứng dụng Android

- Hệ điều hành: Microsoft Windows, Linux và Mac.
- **Java Development Kit (JDK)**: bộ công cụ phát triển cho Java
- **Android Software Development Kit (SDK)**: bộ công cụ phát triển cho Android. Cung cấp bộ thư viện cho phép build, debug các ứng dụng Android
- **Android Studio**: ứng dụng hỗ trợ lập trình thông qua giao diện

Cài đặt

► Java Development Kit (JDK): Download tại đây:

<https://www.oracle.com/java/technologies/javase-downloads.html>

Linux ARM 64 Compressed Archive	157.01 MB	jdk-15.0.1_linux-aarch64_bin.tar.gz
Linux x64 Debian Package	154.79 MB	jdk-15.0.1_linux-x64_bin.deb
Linux x64 RPM Package	162.02 MB	jdk-15.0.1_linux-x64_bin.rpm
Linux x64 Compressed Archive		jdk-15.0.1_linux-x64_bin.tar.gz
macOS Installer		jdk-15.0.1_macos-x64_bin.dmg
macOS Compressed Archive		jdk-15.0.1_macos-x64_bin.tar.gz
Windows x64 Installer		jdk-15.0.1_windows-x64_bin.exe
Windows x64 Compressed Archive	179.27 MB	jdk-15.0.1_windows-x64_bin.zip

You must accept the [Oracle Technology Network License Agreement for Oracle Java SE](#) to download this software.

☒ I reviewed and accept the Oracle Technology Network License Agreement for Oracle Java SE

Download jdk-15.0.1_windows-x64_bin.exe [↓](#)

Cài đặt

- ANDROID STUDIO (IDE): Download tại đây:

<https://developer.android.com/studio/index.html>



Android Studio provides the fastest tools for building apps on every type of Android device.

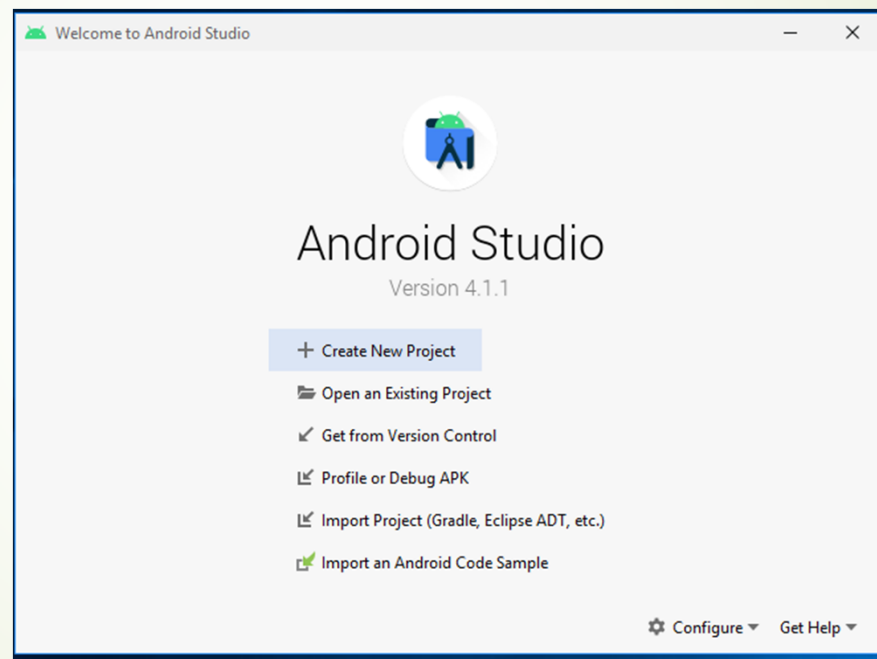
DOWNLOAD ANDROID STUDIO

4.1.1 for Windows 64-bit (896 MB)

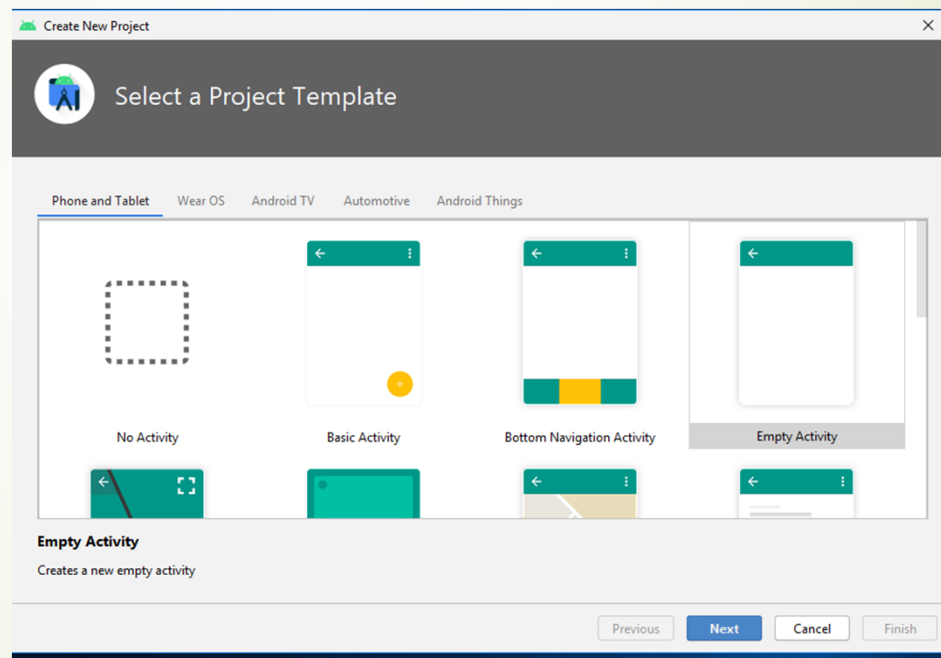
DOWNLOAD OPTIONS

RELEASE NOTES

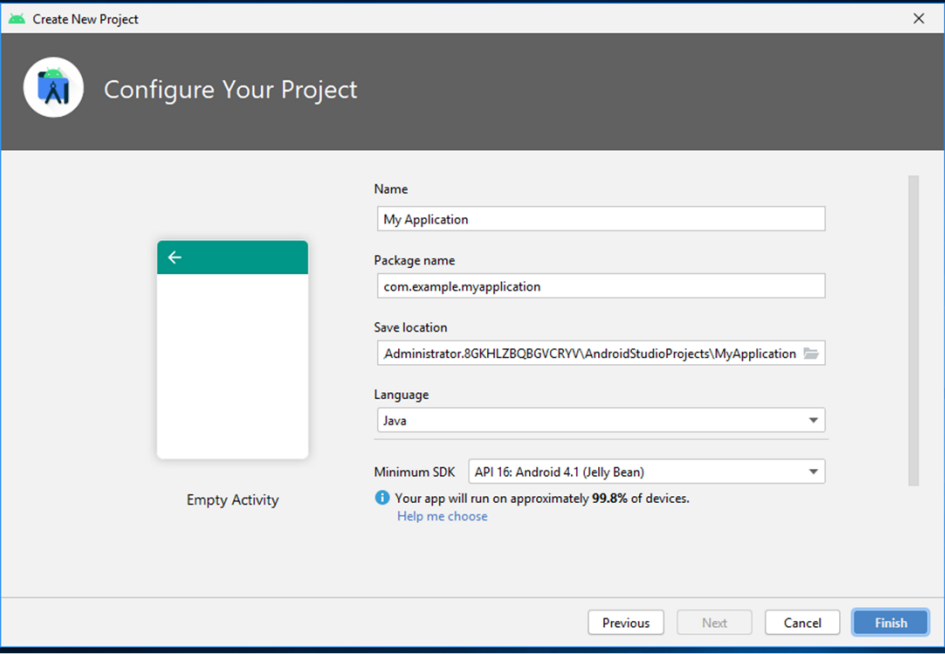
Tạo 1 project mới



Tạo 1 project mới



Tạo 1 project mới



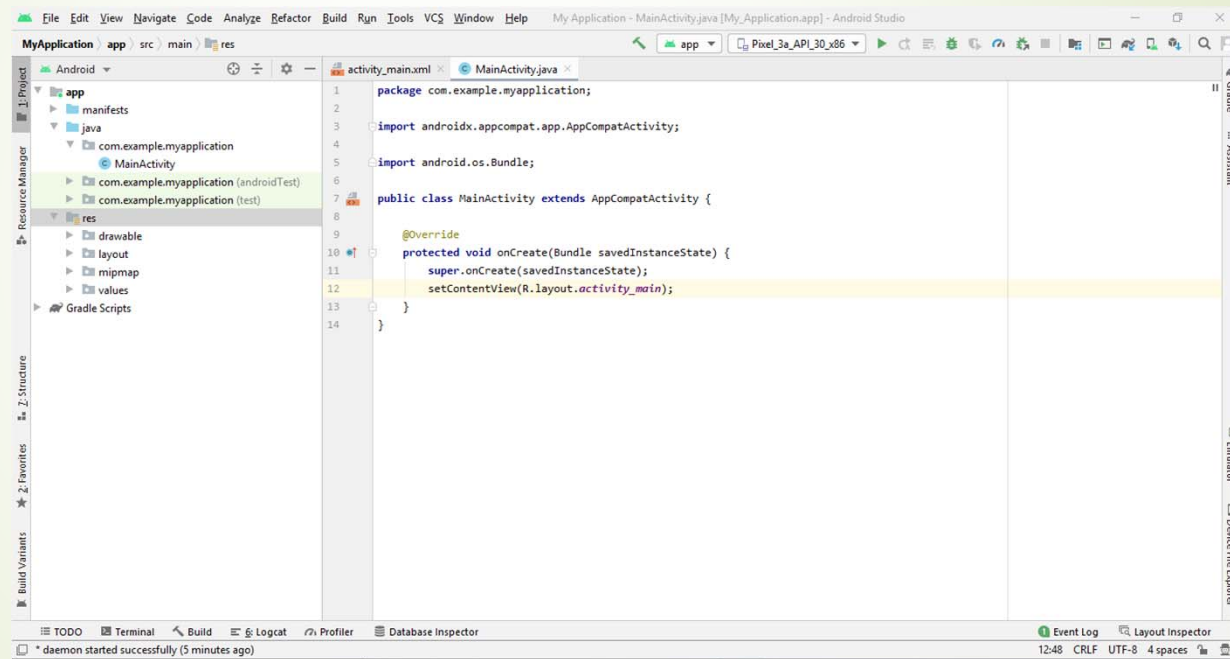
The screenshot shows the 'Create New Project' dialog in Android Studio. The title bar says 'Create New Project'. The main header is 'Configure Your Project' with an Android logo. On the left, there is a preview of an 'Empty Activity' with a green header bar and a white body. On the right, there are several input fields and dropdowns:

- Name:** My Application
- Package name:** com.example.myapplication
- Save location:** Administrator.8GKHLZBQBGVCRYV\AndroidStudioProjects\MyApplication
- Language:** Java
- Minimum SDK:** API 16: Android 4.1 (Jelly Bean)

Below the 'Minimum SDK' dropdown, there is a blue information icon and the text: 'Your app will run on approximately 99.8% of devices.' with a link 'Help me choose'.

At the bottom right, there are four buttons: 'Previous', 'Next', 'Cancel', and 'Finish'.

Tạo 1 project mới



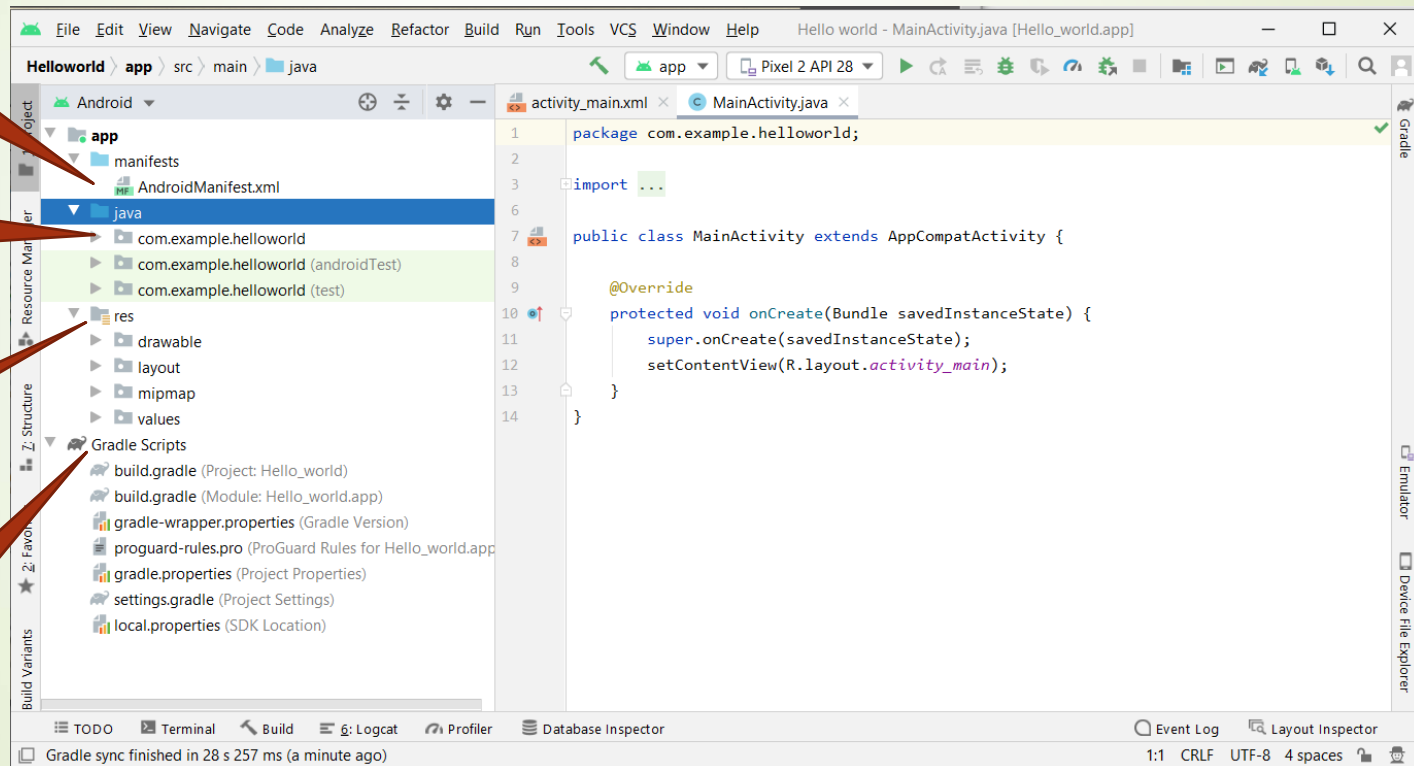
Cấu trúc của một Project

File chứa thông tin, cấu hình của ứng dụng

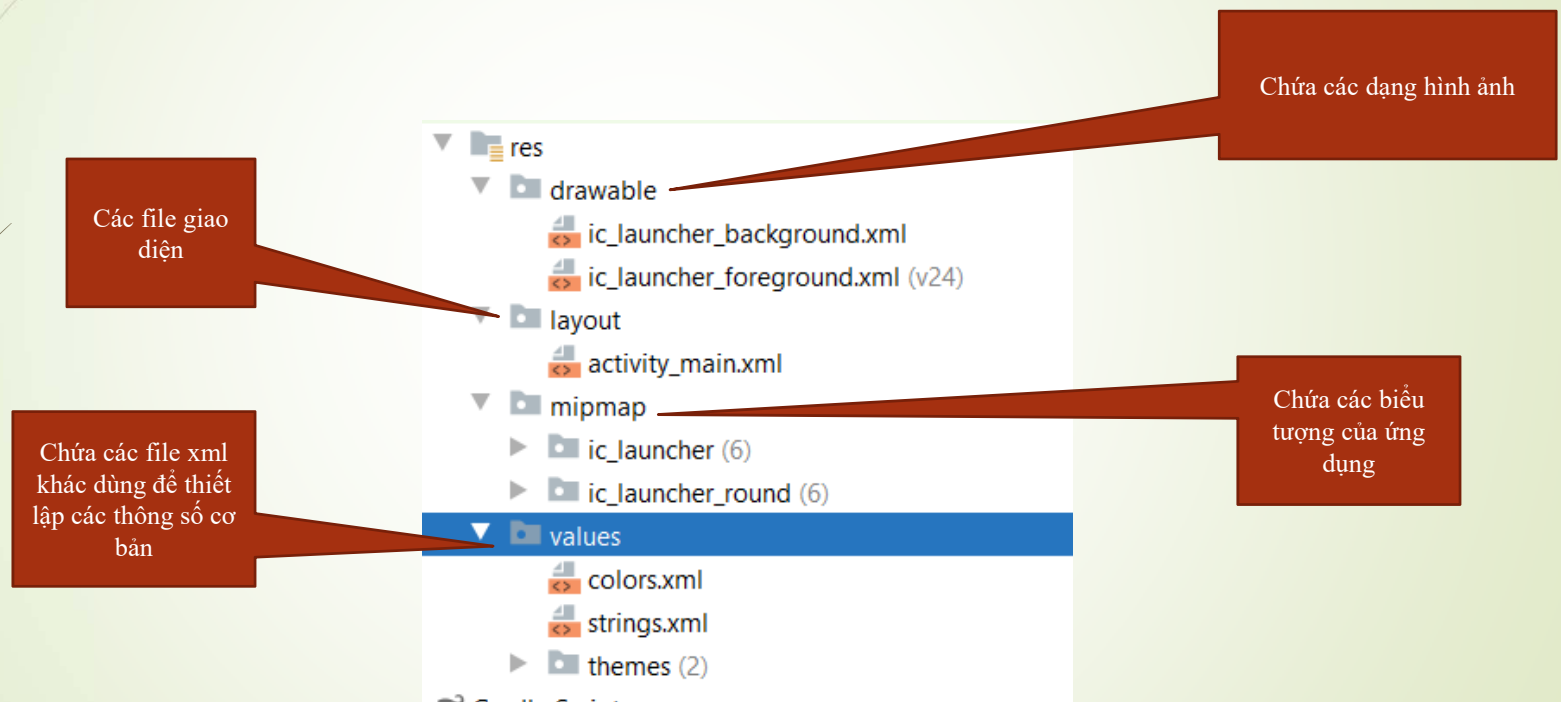
Các file mã nguồn java

Chứa các dữ liệu ngoại trừ code dùng trong ứng dụng

Công cụ build hệ thống được tự động tạo ra do AS điều khiển

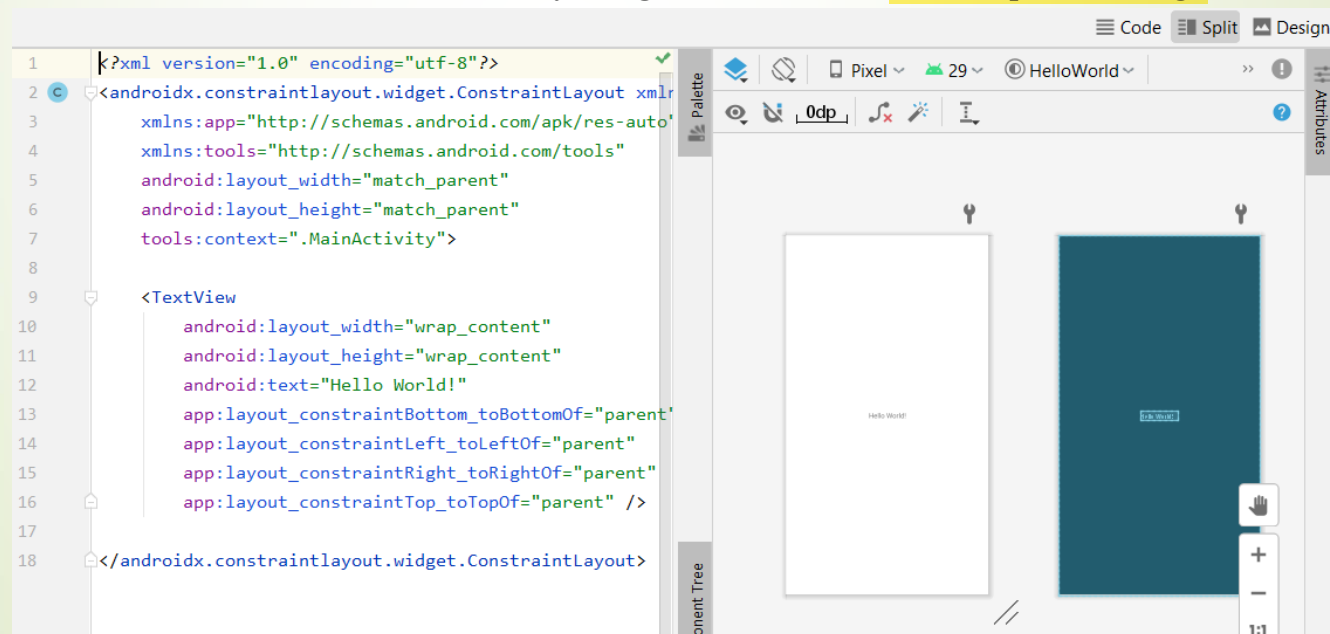


Cấu trúc của một Project



Cấu trúc của một Project

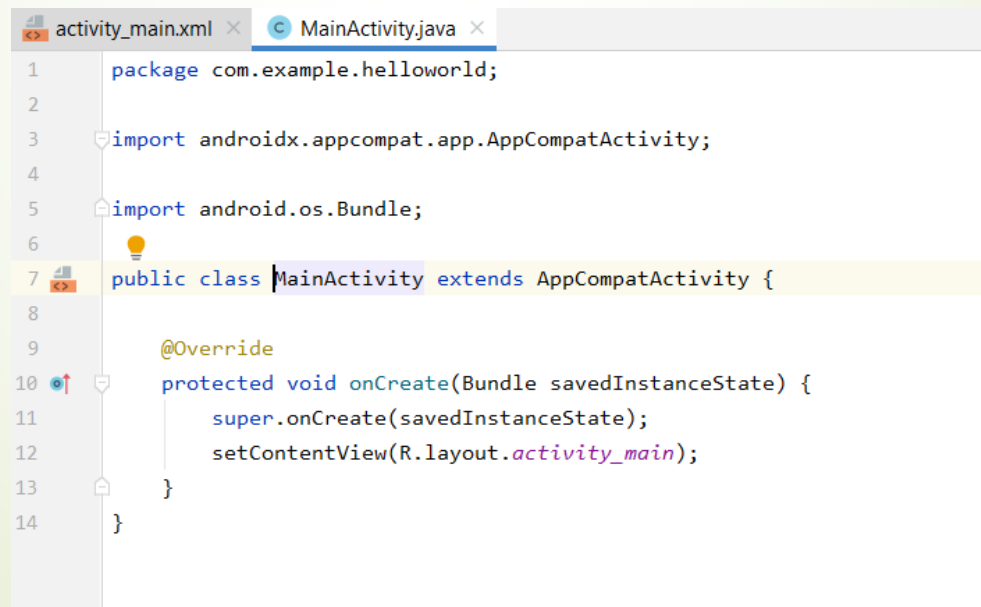
- File activity_main.xml
 - Giao diện chính của ứng dụng sẽ được thiết kế ở đây
 - Có 3 chế độ làm việc với file này trong Android Studio: **Code, Split và Design**



Cấu trúc của một Project

- File MainActivity.java:

- Đây là file code activity chính
- Chứa các lệnh để xử lý các activity liên quan trong ứng dụng



```
1 package com.example.helloworld;
2
3 import androidx.appcompat.app.AppCompatActivity;
4
5 import android.os.Bundle;
6
7 public class MainActivity extends AppCompatActivity {
8
9     @Override
10    protected void onCreate(Bundle savedInstanceState) {
11        super.onCreate(savedInstanceState);
12        setContentView(R.layout.activity_main);
13    }
14 }
```

Cấu trúc của một Project

■ File AndroidManifest.xml:

- Bất kỳ thành phần nào mà bạn muốn phát triển như một phần của ứng dụng đều phải được khai báo ở đây
- Hoạt động như một giao diện giữa Android OS và ứng dụng của bạn nên nếu không khai báo, OS sẽ không ghi nhận
- Ví dụ: cung cấp các quyền truy cập bộ nhớ, danh bạ, internet,...

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3          package="com.example.helloworld">
4
5      <application
6          android:allowBackup="true"
7          android:icon="@mipmap/ic_launcher"
8          android:label="@string/app_name"
9          android:roundIcon="@mipmap/ic_launcher_round"
10         android:supportRtl="true"
11         android:theme="@style/Theme.HelloWorld">
12         <activity android:name=".MainActivity">
13             <intent-filter>
14                 <action android:name="android.intent.action.MAIN" />
15
16                 <category android:name="android.intent.category.LAUNCHER" />
17             </intent-filter>
18         </activity>
19     </application>
20
21 </manifest>
```

Cấu trúc của một Project

▀ File strings.xml

- ▀ Chứa các text mà ứng dụng sử dụng
- ▀ Ví dụ: tên nhãn, nút, chuỗi mặc định,...
- ▀ Bạn có thể đặt các chuỗi này trong các thành phần, tuy nhiên tốt nhất là đặt trong file strings.xml này để quản lý hiệu quả hơn.

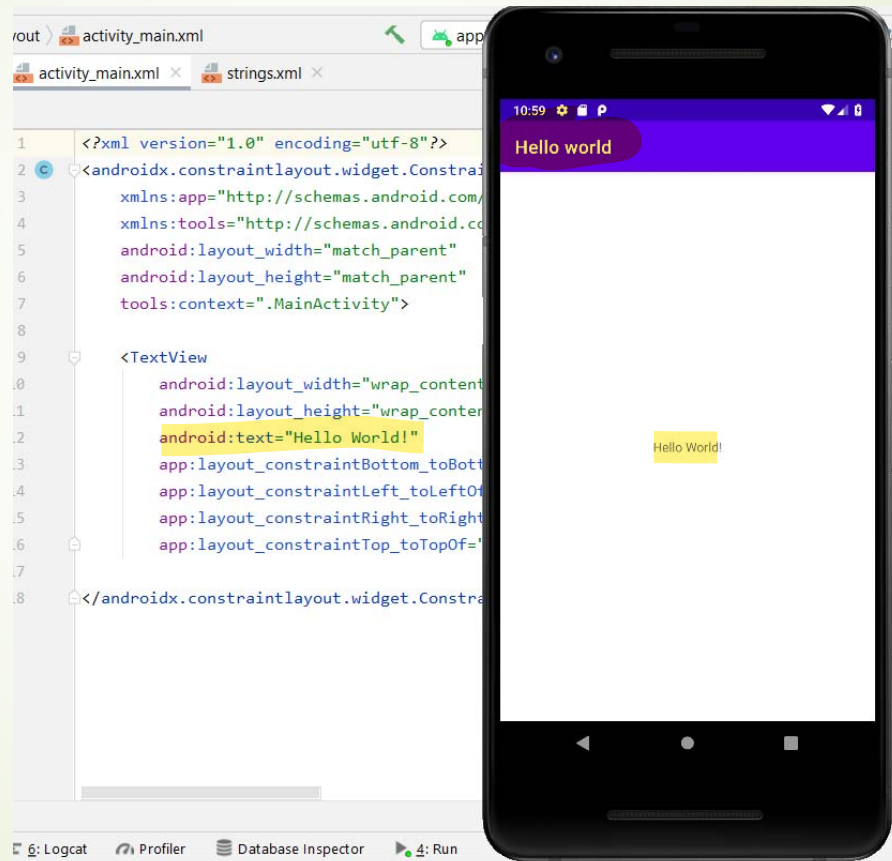
```
1 <resources>
2   <string name="app_name">Hello world</string>
3 </resources>
```

Cấu trúc của một Project

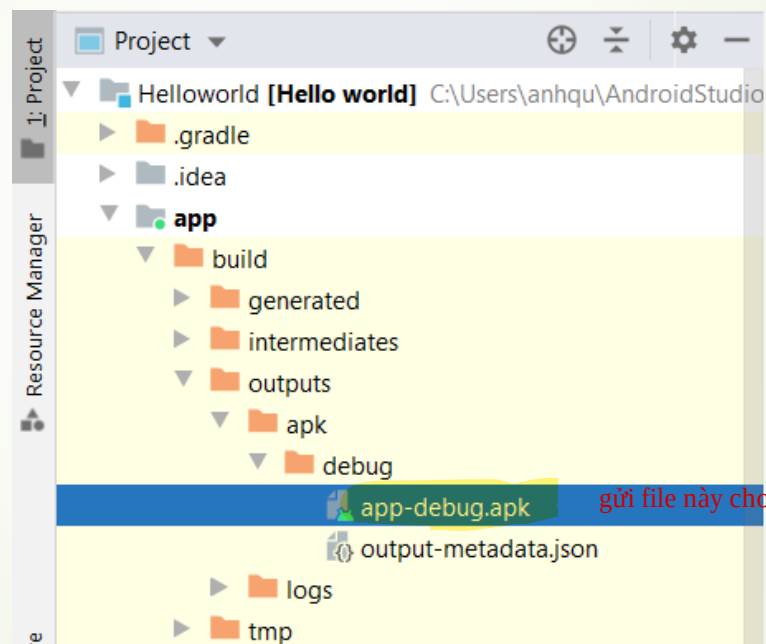
- Cách tham chiếu các tài nguyên khác nhau giữa java và xml:

Resource	Reference in Java	Reference in XML
res/layout/main.xml	R.layout.main	@layout/main
res/drawable-hdpi/icon.png	R.drawable.icon	@drawable/icon
@+id/home_button	R.id.home_button	@id/home_button
<string name = "hello">	R.string.hello	@string/hello

Cấu trúc của một Project



Đóng gói project



gửi file này cho người khác để họ cài đặt



Các thành phần của một ứng dụng

- Một ứng dụng có thể có các thành phần sau:
 - **Activity**
 - **View**
 - **Service**
 - **Broadcast Receiver**
 - **Intent**
 - **Content Provider**
 - **Notification**

Các thành phần của một ứng dụng

➤ Activity:

- Đây là thành phần **quan trọng nhất** của một ứng dụng.
- Mỗi activity thường được trình bày thông qua một giao diện trực quan (cửa sổ)
- Mỗi ứng dụng có thể có **nhiều activity**.
- Các activity kết hợp tạo thành một tổ hợp giao diện người dùng, tuy nhiên chúng **hoạt động độc lập** nhau
- Khi một ứng dụng được khởi động, **activity đầu tiên sẽ kích hoạt hiển thị cho người dùng**
- Để chuyển từ activity này sang activity khác, ta dùng **intent**

Các thành phần của một ứng dụng

➤ View:

- Cung cấp các khối xây dựng cơ bản cho các thành phần giao diện người dùng
- Ví dụ: button, text field, image view,...

text view



Các thành phần của một ứng dụng

➤ Service:

- Không có giao diện người dùng trực quan
- Chạy ẩn dưới background trong một khoảng thời gian không xác định
- Có thể kết nối đến một service đang diễn ra để tương tác với service đó (và khởi động nếu service chưa chạy)

Các thành phần của một ứng dụng

➡ Broadcast Receiver

- ➡ Đây là thành phần không làm gì ngoài việc **lắng nghe và phản hồi các sự kiện**, trạng thái của hệ thống phát ra thông qua intent
- ➡ **Không có giao diện người dùng** nhưng có thể kích hoạt một activity để phản hồi lại sự kiện mà nó nhận được
- ➡ Ví dụ: thông báo tin nhắn, mail, tắt mở mạng,...
- ➡ Có thể hoạt động ngay cả khi ứng dụng bị tắt đi (sử dụng cùng service)



Các thành phần của một ứng dụng

➤ Intent

- Cho phép chuyển đổi từ activity này sang activity khác
- Các activity được gọi có thể ở bên ngoài ứng dụng: mở camera để chụp ảnh, gọi điện,...

Các thành phần của một ứng dụng

➤ Content Provider

- Là thành phần quản lý truy cập dữ liệu
- Dữ liệu thường lưu trên file hệ thống hoặc trong một CSDL SQLite
- Cung cấp các Phương thức khác nhau để các ứng dụng có thể truy cập dữ liệu từ một ứng dụng khác
- Tuy nhiên, các ứng dụng không gọi các Phương thức này trực tiếp mà thông qua một đối tượng **content resolver** và gọi các phương thức thay thế của nó.
- Ví dụ: Content Resolver cung cấp các phương thức query (), insert (), update () và delete () để truy cập dữ liệu của Content Provider



Các thành phần của một ứng dụng

■ Notification:

- Là một thành phần có giao diện người dùng
- Hiển thị các thông báo cần thiết cho người dùng ở giao diện ngoài ứng dụng



Activity – Vòng đời của một ứng dụng



Activity – Vòng đời của một ứng dụng

- ▶ Mỗi ứng dụng của Android đều chạy trong tiến trình riêng của nó (Với phiên bản máy ảo Dalvik của riêng nó)
- ▶ Mỗi khi có một yêu cầu cần xử lý bởi một thành phần cụ thể:
 - ▶ Android đảm bảo tiến trình ứng dụng của thành phần đang chạy
 - ▶ Và khởi động nó nếu cần thiết
 - ▶ Có sẵn một thể hiện thích hợp của thành phần, tạo thể hiện nếu cần thiết.



Activity – Vòng đời của một ứng dụng

- ▶ Tiến trình của một ứng dụng Android sẽ được khi duy trì (khi một số lệnh của nó cần chạy) cho đến khi:
 - ▶ Nó không còn cần thiết
 - ▶ Hoặc hệ thống cần lấy lại bộ nhớ của nó để cung cấp cho ứng dụng khác



Activity – Vòng đời của một ứng dụng

- Một đặc điểm khác thường và cơ bản của Android là thời gian tồn tại của một quy trình ứng dụng không được kiểm soát trực tiếp bởi chính ứng dụng đó. Thay vào đó, nó được xác định bởi hệ thống thông các vấn đề sau:
 - Các phần của ứng dụng mà hệ thống biết đang chạy,
 - Những điều này quan trọng như thế nào đối với người dùng
 - Và có bao nhiêu bộ nhớ tổng thể có sẵn trong hệ thống.

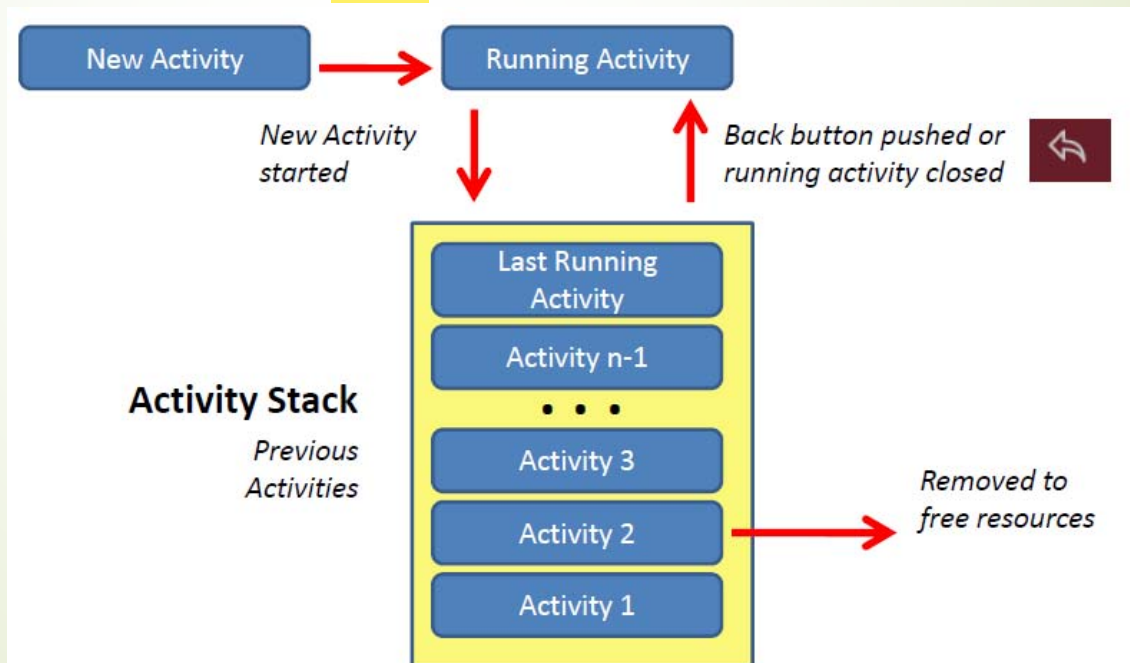
Activity – Vòng đời của một ứng dụng

- Các thành phần của ứng dụng có một vòng đời (lifecycle)
 - Bắt đầu khi Android khởi tạo chúng để phản hồi đến các intent
 - Kết thúc khi các thể hiện bị hủy
 - Ở giữa: đôi khi chúng có thể hoạt động hoặc ngưng hoạt động, hiển thị hoặc ẩn với người dùng



Activity Stack

- ▀ Các Activity của hệ thống được quản lý bởi Activity Stack
- ▀ Hoạt động theo cơ chế **LIFO**

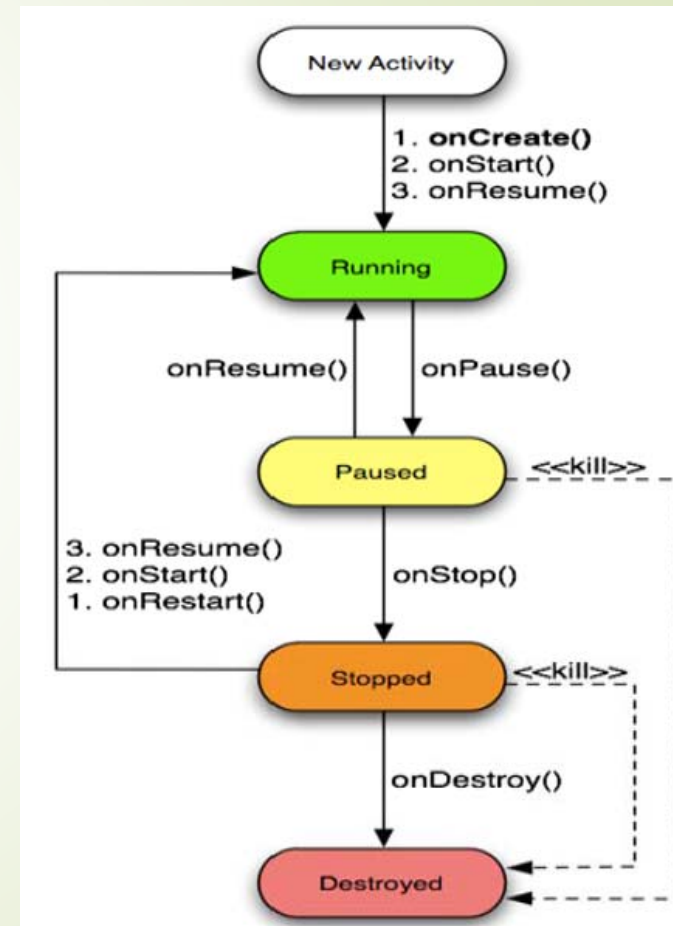


Activity – LifeCycle states

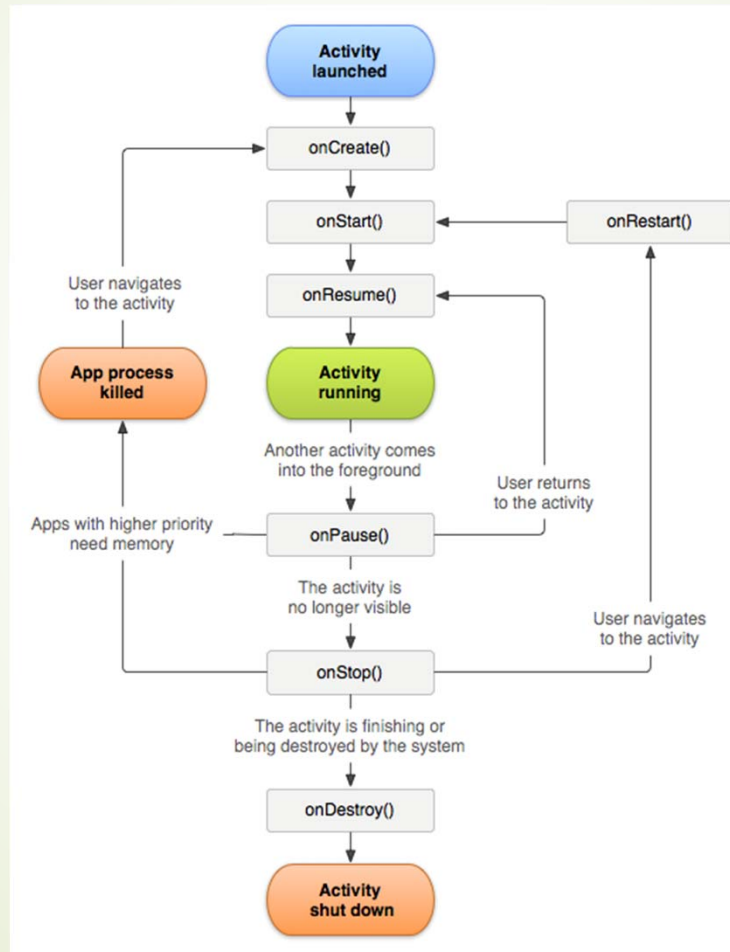
► Mỗi activity thường vòng đời có 3 trạng thái:

một pop-up hiện lên

1. Running: khi nó là foreground của màn hình (trên cùng của activity stack), cho phép user tương tác.
2. Paused: khi activity bị mất focus nhưng user vẫn nhìn thấy (ví dụ activity dạng dialog). Có khả năng bị hệ thống “kill” trong tình huống bộ nhớ quá ít.
3. Stopped: khi activity bị mất focus và user không nhìn thấy (activity mới full màn hình). Có thể bị hệ thống “kill” bất kỳ lúc nào.



LifeCycle của ứng dụng



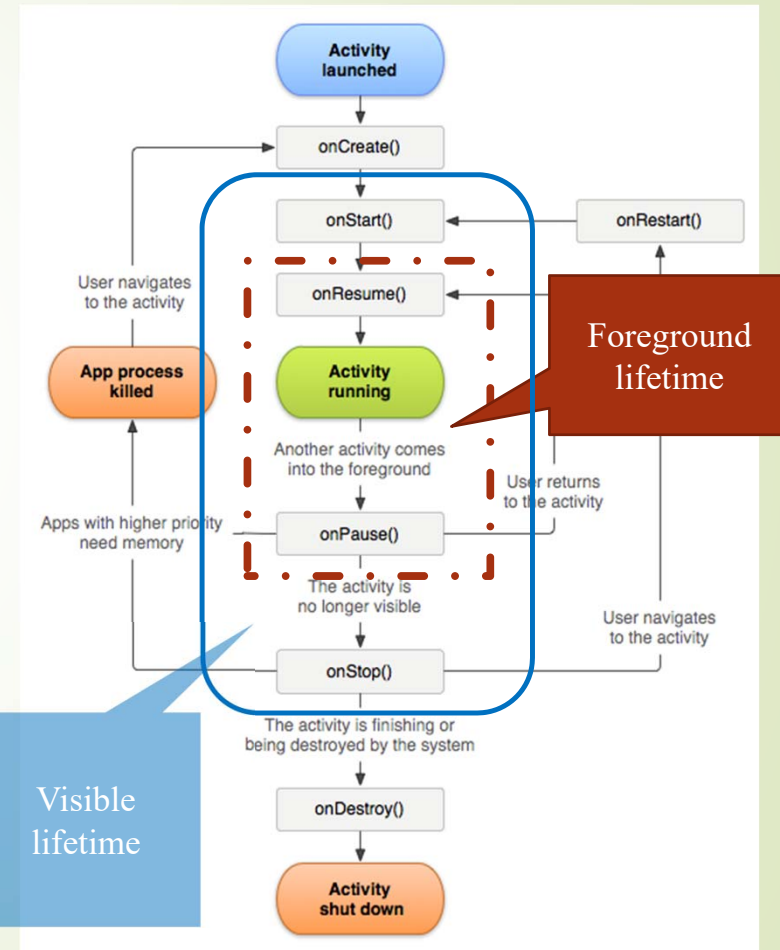
LifeCycle của ứng dụng

Visible lifetime:

- Từ sau khi gọi `onStart()` cho đến `onStop()`: user vẫn thấy activity, mặc dù nó không ở foreground và không tương tác với user
- `onStart()`, `onStop()` có thể gọi nhiều lần vì ẩn hiện activity đối với user
- Giữa 2 Phương thức này ta có thể duy trì các tài nguyên cần thiết để hiển thị activity cho user

Foreground lifetime:

- Từ sau khi gọi `onResume()` cho đến `onPause()`: activity nằm trên cùng và user tương tác được với nó





Thực hành



View và các điều khiển cơ bản

View và các điều khiển cơ bản

➤ View:

➤ Là các đối tượng dùng để trình bày giao diện của ứng dụng.

➤ Có thể tạo View bằng code java hoặc xml.

➤ Một số view cơ bản:

xml dùng xây dựng giao diện cố định theo thời gian

➤ View hiển thị: TextView, ImageView,..

➤ Control nhập liệu: Button, EditText, CheckBox,...

➤ Adapter View: ListView, GridView, Spinner,...

View và các điều khiển cơ bản

Ví dụ:

➤ **Tạo TextView bằng xml:**

khi sử dụng chế độ design thì code xml tự tạo tương ứng, ta chỉ cần chỉnh sửa lại cho phù hợp.

```
<TextView  
    android:id="@+id/txtView"    đối tượng khi chỉ cần hiển thị mà không thao tác  
                                gì khác thì không cần đặt id.  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Hello World!"  
    app:layout_constraintBottom_toBottomOf="parent"  
    app:layout_constraintLeft_toLeftOf="parent"  
    app:layout_constraintRight_toRightOf="parent"  
    app:layout_constraintTop_toTopOf="parent" />
```

- Trong đó, id là thuộc tính cần thiết để tham chiếu đến đối tượng trong java
- Tham chiếu đến TextView trong java:

```
TextView textView=(TextView)findViewById(R.id.txtView);
```

View và các điều khiển cơ bản

Ví dụ:

➡ Tạo TextView bằng java:

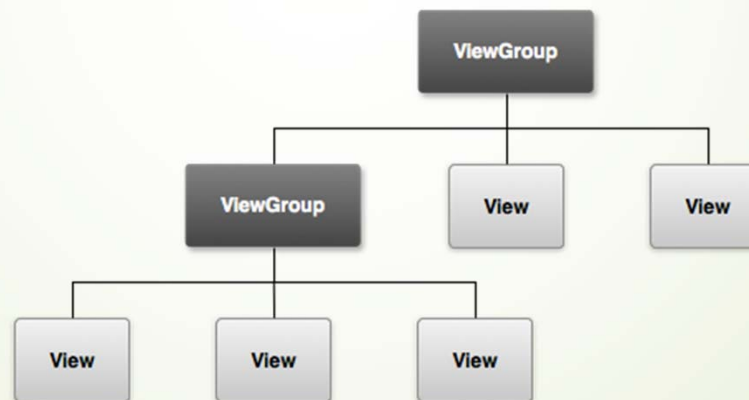
```
ConstraintLayout constraintLayout=(ConstraintLayout)findViewById(R.id.main_layout);  
TextView textView1= new TextView(this);  
textView1.setText("Hello abc!");  
constraintLayout.addView(textView1);
```

để lấy giá trị: getText();

View và các điều khiển cơ bản

➤ ViewGroup:

- Là đối tượng dùng để chứa các View và ViewGroup khác.
- Dùng để trình bày layout cho ứng dụng.
- Tạo ra một hệ thống phân cấp các đối tượng View giúp ta xây dựng những layout phức tạp.

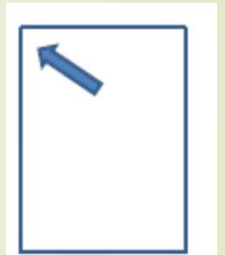


View và các điều khiển cơ bản

Một số layout cơ bản:

➤ **FrameLayout:**

- Đây là layout đơn giản nhất
- Tất cả các phần tử con của layout được ghim lên góc trên cùng bên trái màn hình
- Các bố trí theo từng khung, có nghĩa các thành phần bên có thể chồng lên nhau
- Thứ tự chồng theo thứ tự code của các thành phần: thành phần code sau sẽ được chồng lên thành phần code trước



View và các điều khiển cơ bản

Một số layout cơ bản:

➤ LinearLayout:

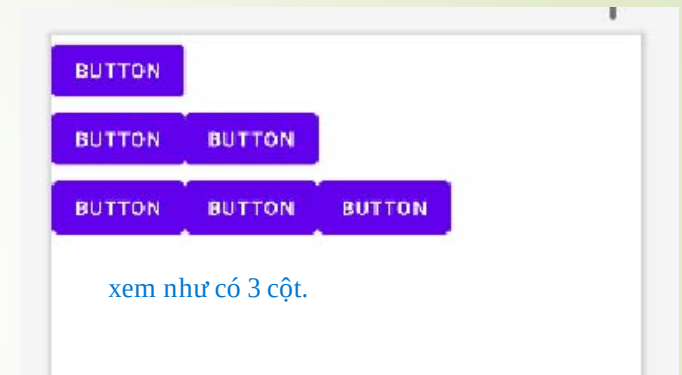
- Canh chỉnh các thành phần con bên trong nó theo một hướng ngang hoặc dọc (nhờ thuộc tính **android:orientation**)
- Chiều dọc (vertically): chỉ có mỗi con trên một hàng, bất kể chúng rộng bao nhiêu
- Chiều ngang (horizontally): chỉ có một hàng, chiều cao của hàng chính là chiều cao của thành phần con cao nhất (kèm padding)
- Có thể sử dụng thuộc tính **margin** để điều chỉnh khoảng cách giữa các thành phần con và vị trí của từng thành phần bằng gravity

View và các điều khiển cơ bản

Một số layout cơ bản:

➤ **TableLayout:**

- Sắp xếp các con của nó thành hàng và cột
- Không hiển thị đường viền
- Mỗi hàng là một đối tượng TableRow
- Mỗi TableRow chứa các view con, mỗi view con như 1 cell
- Số cột chính là số view con nhiều nhất của một TableRow
- Mỗi cell cũng có thể là 1 ViewGroup



View và các điều khiển cơ bản

Một số layout cơ bản:

- **ConstraintLayout:**

- Đây là dạng layout rất mạnh, khuyến khích nên dùng khi user cần xây dựng những layout phức tạp (giảm việc lồng các layout vào nhau)
- Nhờ hỗ trợ các tính năng ràng buộc các thành phần, user có thể sắp xếp, định vị các view con một cách dễ dàng

View và các điều khiển cơ bản

Một số layout cơ bản:

The diagram illustrates basic layout constraints in Android Studio. It shows two views: a design view on the left and a code view on the right. The design view shows a button and a text field with various constraints. The code view shows the corresponding XML code for the button, including attributes like `layout_width`, `layout_height`, `layout_constraintHorizontal_bias`, `layout_constraintVertical_bias`, `layout_constraintTop_toBottomOf`, `layout_constraintBottom_toBottomOf`, `layout_constraintEnd_toEndOf`, `layout_constraintStart_toStartOf`, `layout_marginTop`, `id`, and `text`. A red circle labeled "Bias" points to the bias attributes in the code view.

Declared Attributes

Attribute	Value
<code>layout_width</code>	<code>wrap_content</code>
<code>layout_height</code>	<code>wrap_content</code>
<code>layout_constraintHorizontal_bias</code>	0.191
<code>layout_constraintVertical_bias</code>	0.135
<code>layout_constraintTop_toBottomOf</code>	<code>@+id/editTextTextPersonName</code>
<code>layout_constraintBottom_toBottomOf</code>	<code>parent</code>
<code>layout_constraintEnd_toEndOf</code>	<code>parent</code>
<code>layout_constraintStart_toStartOf</code>	<code>parent</code>
<code>layout_marginTop</code>	148dp
<code>id</code>	<code>button8</code>
<code>text</code>	<code>Button</code>

Layout

Constraint Widget

View và các điều khiển cơ bản

- Một layout có thể được gọi bởi ứng dụng bằng cú pháp sau:

```
setContentView(R.layout.ten_layout);
```

- Ví dụ, để gọi layout **activity_main.xml**, **ten_layout** sẽ là : **activity_main**



Thực hành

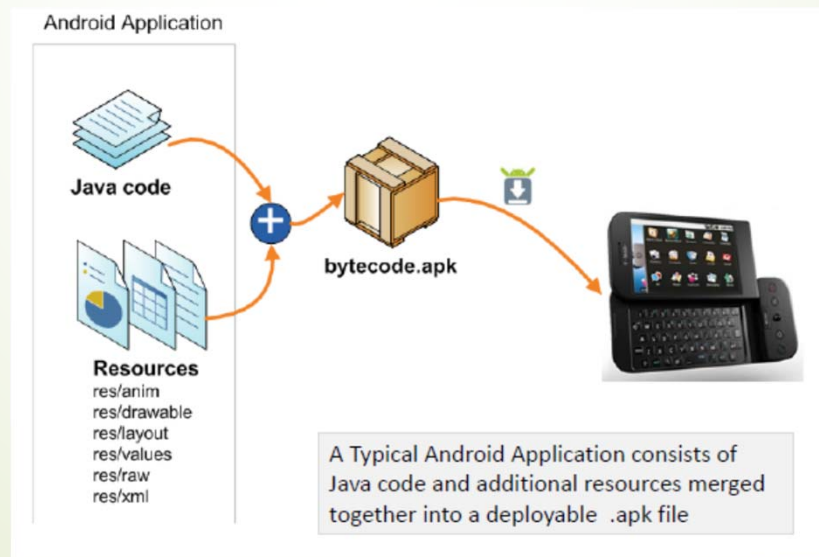


Tài nguyên ứng dụng cơ bản và Intent

Tài nguyên ứng dụng cơ bản

Định nghĩa:

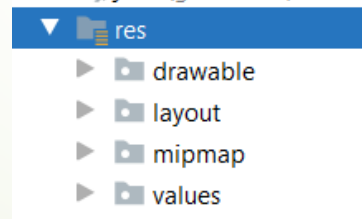
- Tài nguyên là các file bên ngoài (không phải file code) được sử dụng bởi code của bạn và được biên dịch vào ứng dụng khi được build
- Android hỗ trợ một số file tài nguyên khác nhau, chẳng hạn như xml, png, jpg,..



Tài nguyên ứng dụng cơ bản

Copy / Paste tài nguyên:

- Bạn sẽ tạo và chứa các file tài nguyên trong thư mục **res** của project
- Các file tài nguyên này sẽ được biên dịch vào file **APK**
- **Android tạo một lớp wrapper, được gọi là R, bạn có thể sử dụng để tham chiếu đến các tài nguyên này trong code của mình. R chứa các lớp con được đặt tên theo đường dẫn và tên file của file nguồn**



Tài nguyên ứng dụng cơ bản

Thư mục	Loại tài nguyên
res/drawable/	Các file *.png, *.jpg
res/layout/	Các file xml dùng để xây dựng giao diện người dùng ứng dụng
res/mipmap/	Tương tự drawable, chứa các hình ảnh bitmap, nhưng được sử dụng đặc biệt cho các biểu tượng launcher của ứng dụng.
res/values/	Các file xml mà có thể biên dịch thành nhiều loại tài nguyên colors.xml: định nghĩa các màu sắc dùng trong ứng dụng (giá trị dạng hex) strings.xml: định nghĩa các giá trị dạng chuỗi themes/ chứa các file themes.xml dùng để định nghĩa các style của đối tượng
res/menu/	Chứa các tập tin xml dùng để xây dựng menu
res/raw/	Các tập tin tùy ý được sao chép trực tiếp vào ứng dụng. Được thêm vào file nén (không biên dịch) do ứng dụng tạo ra
res/xml/	Chứa các tập tin xml tùy ý, được sử dụng bởi Android cho các tác vụ khác nhau

Tài nguyên ứng dụng cơ bản

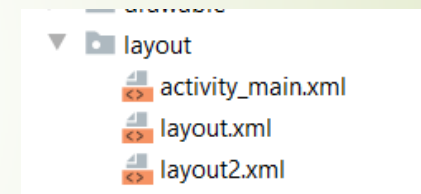
Truy cập tài nguyên trong java:

➤ **Hiển thị layout màn hình:**

```
setContentView(R.layout.activity_main);
```

```
setContentView(R.layout.layout);
```

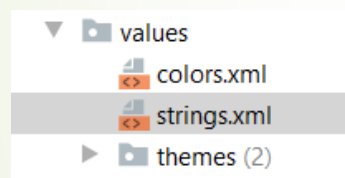
```
setContentView(R.layout.layout2);
```



Tài nguyên ứng dụng cơ bản

Truy cập tài nguyên trong java:

➤ Truy xuất tài nguyên **string** từ **res/value/strings.xml**:



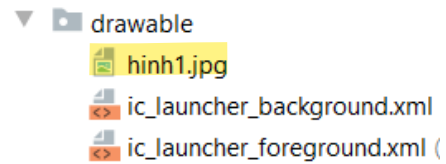
```
<resources>
    <string name="app_name">Hello world</string>
    <string name="test1">Hello xyz</string>
    <string name="test2">Hello 123</string>
</resources>
```

String tam1=this.getString(R.string.test1);

Tài nguyên ứng dụng cơ bản

Truy cập tài nguyên trong java:

➤ Truy xuất một hình ảnh trong **res/drawable/**:



```
imageView.setImageResource(R.drawable.hinh1);
```



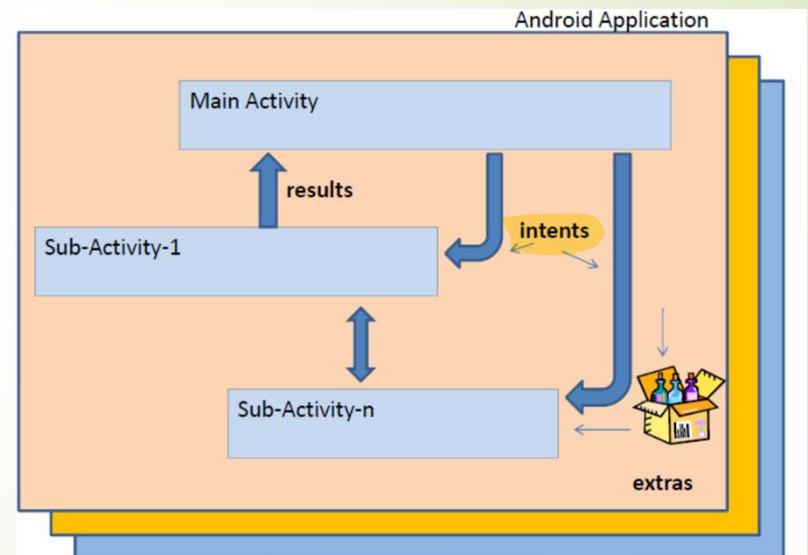
Thực hành



Intent

Intent – Cơ chế hoạt động

Dùng để truyền tải thông điệp, yêu cầu một hành động xử lý từ thành phần được gọi.



Intent – Cơ chế hoạt động

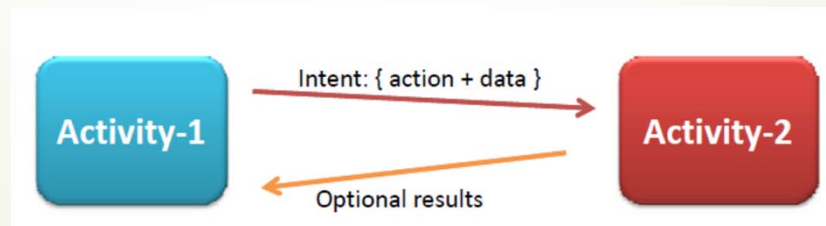
Intent được dùng bởi các trường hợp sau:

<code>startActivity(intent)</code>	Khởi động một Activity
<code>sendBroadcast(intent)</code>	Chuyển thông điệp đến BroadcastReceiver
<code>startService(intent) ()</code> hoặc <code>bindService(intent, ...)</code>	Giao tiếp với Service

Intent – Các dạng intent

Có 2 loại intent:

- **Explicit Intent:** chỉ rõ thành phần xử lý thông qua tên lớp, dùng để gọi đến các thành phần trong cùng ứng dụng
- **Implicit Intent:** không chỉ rõ thành phần xử lý, thay vào đó cần mô tả Action và Data



Intent – Xây dựng intent

➤ Explicit Intent: *tường minh*

➤ Gọi đến 1 component trong ứng dụng:

```
Intent intent=new Intent(this,<Component>);  
startActivity(intent);
```

➤ Ví dụ:

```
Intent intent=new Intent(this,MainActivity2.class);  
startActivity(intent);
```

Intent – Xây dựng intent

➤ Implicit Intent:

```
Intent myActivity = new Intent (action, data);  
startActivity (myActivity);
```

Built-in
or
user-created
activity

Primary data (as an URI)
tel://
http://
sendto://

Intent – Xây dựng intent

➤ Implicit Intent:

sử dụng intent để móc nối đến một ứng dụng khác trên điện thoại

```
Intent myActivity2 = new Intent (Intent.ACTION_DIAL,  
Uri.parse( "tel:555-1234" ));  
startActivity(myActivity2);
```



Intent – Xây dựng intent

(một cách khác sử dụng không tường minh)

➤ Implicit Intent:

```
Intent myActivity2=new Intent(action);  
myActivity2.setData(data);  
startActivity(myActivity2);
```



```
Intent myActivity2=new Intent(Intent.ACTION_DIAL);  
myActivity2.setData(Uri.parse("tel:555-1234"));  
startActivity(myActivity2);
```

Intent – Xây dựng intent

► Implicit Intent:

► Ví dụ một số cặp Action/Data

Action	Data
ACTION_DIAL	tel:123
ACTION_VIEW	http://www.google.com
ACTION_EDIT	content://contacts/people/2

Intent – Xây dựng intent

► Implicit Intent:

List of standard actions that Intents can use for launching activities (usually through `startActivity(Intent)`).

ACTION_MAIN	ACTION_ANSWER
ACTION_VIEW	ACTION_INSERT
ACTION_ATTACH_DATA	ACTION_DELETE
ACTION_EDIT	ACTION_RUN
ACTION_PICK	ACTION_SYNC
ACTION_CHOOSER	ACTION_PICK_ACTIVITY
ACTION_GET_CONTENT	ACTION_SEARCH
ACTION_DIAL	ACTION_WEB_SEARCH
ACTION_CALL	ACTION_FACTORY_TEST
ACTION_SEND	
ACTION_SENDTO	

For a list of actions see:

<http://developer.android.com/reference/android/content/Intent.html>

Intent – Xây dựng intent

gửi dữ liệu

➤ Extras: Có 2 cách gửi dữ liệu vào Intent:

➤ Trực tiếp: dùng `putExtra(Key, Value)` thiết lập vào Intent

```
Intent intent=new Intent(this,MainActivity2.class);  
intent.putExtra("test","Hello");  
startActivity(intent);
```

➤ Bundle:

➤ Tạo đối tượng Bundle, dùng `set<KDL>(Key, Value)` vào đối tượng đó

➤ Dùng `putExtra()` gửi Bundle vào Intent

```
Intent intent=new Intent(this,MainActivity2.class);  
Bundle bundle=new Bundle();  
bundle.putString("test2","123");  
intent.putExtras(bundle);  
startActivity(intent);
```

Intent – Truy xuất dữ liệu từ intent

nhận dữ liệu

➤ Trực tiếp:

dùng `get<KDL>Extra(Key, DefaultValue)` để truy xuất dữ liệu

```
Intent intent=getIntent();  
String tam=intent.getStringExtra("test");
```

➤ Bundle:

➤ Dùng `getExtras()` để truy xuất đối tượng Bundle từ Intent

➤ Dùng `get<KDL>(Key, DefaultValue)` để truy xuất dữ liệu trong Bundle

```
Intent intent=getIntent();  
Bundle bundle=intent.getExtras();  
String tam=bundle.getString("test2");
```

Intent – Gửi và phản hồi Intent trong Activity

- Muốn nhận kết quả trả về từ activity được gọi, ta sử dụng:

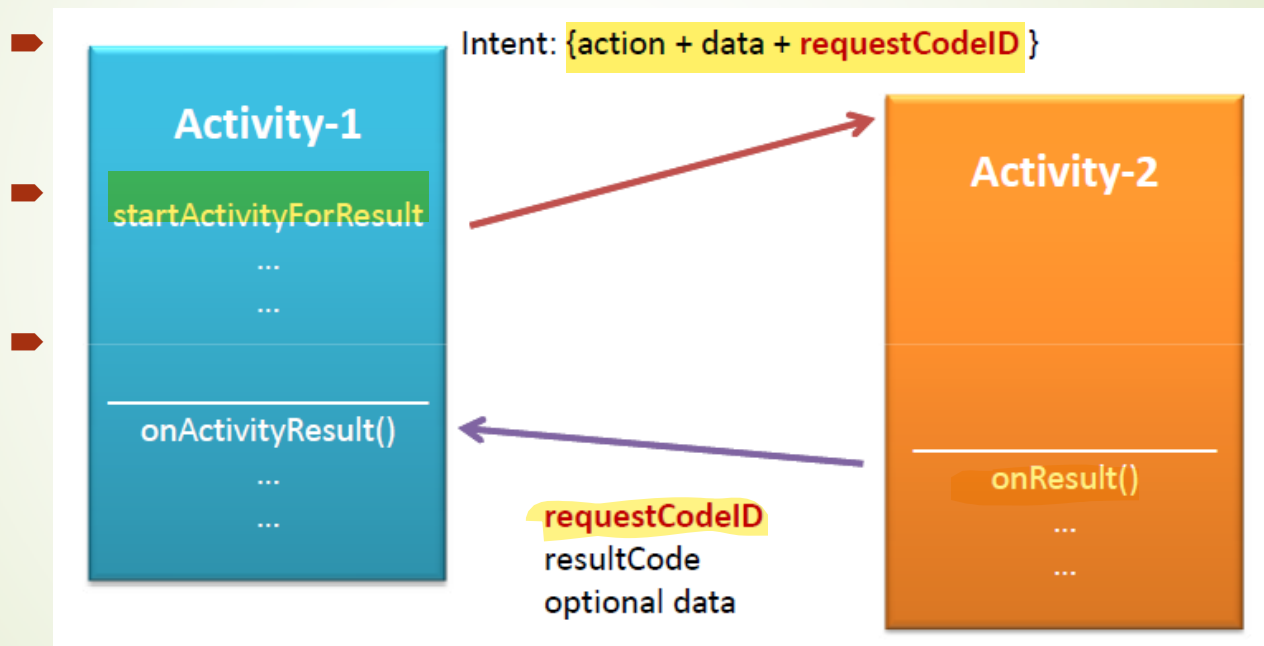
startActivityForResult(Intent, requestCodeID)

- Kết quả được trả về theo Phương thức:

onActivityResult(requestCodeID, resultCode, Intent)

- Trước khi một activity thoát, nó có thể gọi **setResult (resultCode)** để trả về 1 tín hiệu kết thúc cho activity gọi nó.
- resultCode: **Activity.RESULT_CANCELED, Activity.RESULT_OK**

Intent – Gửi và phản hồi Intent trong Activity



trả về 1



Intent Filter



Intent Filter – Mô tả

- Mô tả cấu trúc Intent và chỉ nhận các Intent theo đúng cấu trúc đã mô tả
- Có thể lọc Intent theo 3 thuộc tính: Action, Data và Category

Intent Filter – Quy tắc thiết lập

- ▶ Lọc Intent theo thứ tự ưu tiên khi có nhiều thuộc tính được thiết lập và có những quy tắc nhất định:
 - ▶ Nếu không thiết lập Action, chỉ nhận các Intent không có Action.
 - ▶ Nếu thiết lập Action và không thiết lập thuộc tính Data, chỉ cho phép lọc các Intent không Data
 - ▶ IntentFilter cho phép nhận các Intent có bất kỳ dữ liệu nào có liên quan thuộc tính Action

Intent Filter – Xây dựng

- Bằng java:
 - Dùng `IntentFilter()`
- Bằng `AndroidManifest.xml`:
 - Sử dụng cặp thẻ `<intentfilter><intenfilter/>`
 - Trong cặp thẻ trên có thể chứa: `<action>`, `<data>`, `<category>`

Intent Filter – Xây dựng

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="cis493.intentfilters" android:versionCode="1" android:versionName="1.0.0">
    <uses-permission android:name="android.permission.RECEIVE_SMS" />
    <application android:icon="@drawable/icon" >
        <activity android:name=".FancySms" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <receiver android:name="SMSReceiver" android:enabled="true" >
            <intent-filter>
                <action android:name="android.provider.Telephony.SMS_RECEIVED" />
            </intent-filter>
        </receiver>
    </application>
</manifest>
```



Thực hành



Asset – SharedReference – Bộ nhớ thiết bị

Asset

lưu localStorage (trên máy cục bộ của người dùng) thay vì đẩy dữ liệu lên server.

- Đây là thư mục chứa các tập tin không biên dịch của project
- Dữ liệu được chép vào thư mục này trước, chỉ cho phép đọc ra, không ghi vào được
 - Truy xuất đối tượng: `getAsset()`
 - Các Phương thức xử lý chính: `list(String path)`, `open(String filename, int mode)`, `close()`
 - Các chế độ mở: `ACCESS_BUFFER`, `ACCESS_RANDOM`, `ACCESS_STREAMING`, `ACCESS_UNKNOWN`
 - Truy xuất font thông qua Asset: `createFromAsset`

```
Typeface arial = Typeface.createFromAsset(getAssets(), "font/arial.ttf");  
textView.setTypeface(arial);
```

SharedReference

- ▶ Lưu trữ:

- ▶ Cho phép lưu trữ và truy xuất theo cặp khóa Key- Value cho các kiểu dữ liệu cơ bản: Boolean, Float, Int, Long, String
- ▶ Dữ liệu vẫn được bảo toàn ngay ứng dụng bị đóng hoàn toàn
- ▶ Preference gần giống với Bundle nhưng nó có thể lưu trữ dữ liệu còn Bundle thì không.

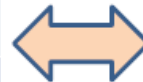
SharedPreferences

- ▶ Truy xuất:
 - ▶ `getDefaultSharedPreferences()`: Lấy một SharedPreferences mặc định trong ứng dụng
 - ▶ `getSharedPreferences()`: lấy một SharedPreferences bằng cách truyền vào tên tập tin đã có
 - ▶ `getPreferences()`: lấy một Preference mặc định tương ứng với Activity thực thi
- ▶ Mặc định SharedPreferences được lưu theo đường dẫn:
`data/data/<package_Name_app>/shared_prefs/<filename_>.xml`

SharedPreferences

Xxx = { Long, Int, Double, Boolean, String }

Preference Container	
Key	Value



E
D
I
T
O
R

.getXxx(key_n)
.getAll()
.getStringSet()
...

.putXxx(key_n, value_n)

.remove(key_n)
.clear()
.commit()

Bộ nhớ thiết bị

- ▶ Bộ nhớ trong:
 - ▶ Được cấp phát khi ứng dụng được cài đặt trên thiết bị.
 - ▶ Bộ nhớ của ứng dụng chỉ do chính ứng dụng đó truy xuất
 - ▶ Các tập tin được lưu trữ ở: `/data/data/<package_name_app>/files`
 - ▶ Phương thức truy xuất: `getFilesDir()`

Bộ nhớ thiết bị

- ▶ Bộ nhớ trong:

- ▶ Truy xuất ghi tập tin:

- ▶ Dùng `openFileOutput()` nhận về dữ liệu stream cho đối tượng `FileOutputStream`
 - ▶ Ghi dữ liệu với `write()`
 - ▶ Đóng stream bằng `close()`

```
String hello = "Hello Android!";  
FileOutputStream fos = openFileOutput("Hello_file.txt",  
                                     MODE_APPEND);  
  
fos.write(hello.getBytes());  
fos.close();
```

Bộ nhớ thiết bị

- ▶ Bộ nhớ trong:

- ▶ Truy xuất đọc tập tin:

- ▶ Dùng `openFileInput()` nhận về dữ liệu stream cho đối tượng `FileInputStream`
 - ▶ Đọc dữ liệu với `read()`
 - ▶ Đóng stream bằng `close()`

```
FileInputStream fis=openFileInput("Hello_file.txt");  
int size=fis.available();  
byte[] buffer=new byte[size];  
fis.read(buffer);  
String s=new String(buffer);  
fis.close();
```

Bộ nhớ thiết bị

- Bộ nhớ ngoài: cần cấp quyền truy xuất bộ nhớ này

```
<uses-permission  
    android:name="android.permission.WRITE_EXTERNAL_STORAGE">  
</uses-permission>
```

- Đường dẫn lưu trữ: /mnt/sdcard/Android/<package_name>/files
- Truy xuất bằng `getExternalFilesDir(filepath)`
- Nên kiểm tra tình trạng bộ nhớ ngoài trước khi truy xuất bằng `getExternalStorageState()`

```
public boolean isExternalStorageWritable() {  
    String state = Environment.getExternalStorageState();  
    if( Environment.MEDIA_MOUNTED.equals(state))  
        return true;  
    return false;  
}
```



Thực hành



Adapter & AdapterView

Khái niệm

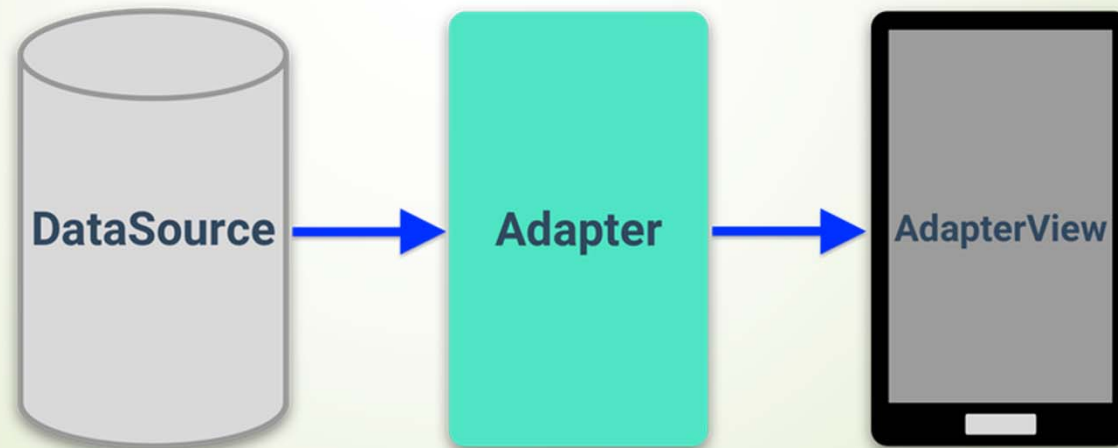
- AdapterView:
 - Là đối tượng điều khiển dạng tập hợp
 - Hiển thị thông tin theo dạng danh sách
 - Thực hiện quản lý thông tin theo từng mục riêng biệt
- Các Phương thức quan trọng dùng trên AdapterView:
 - `setOnClickListener()`
 - `setOnItemSelectedListener()`



Khái niệm

■ Adapter:

- Là đối tượng cho phép gắn kết dữ liệu lên AdapterView
- Quản lý giao diện, số lượng chỉ mục trên AdapterView
- Thực hiện truy vấn dữ liệu, sắp xếp dữ liệu





Các dạng Adapter

- ListView
- GridView
- ExpandableListView
- Gallery
- StackView
- Spinner
- RecyclerView.

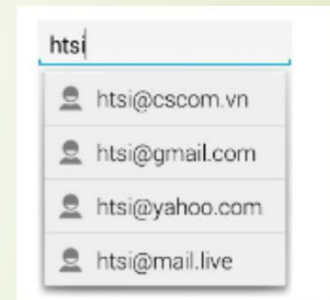


Các dạng Adapter

- ▀ Các Phương thức xử lý quan trọng trên Adapter:
 - ▀ getCount()
 - ▀ getItem()
 - ▀ getItemId()
 - ▀ getView()

AutoCompleteTextView

- Đối tượng kế thừa từ EditText
- Cho phép xây dựng dữ liệu mẫu hỗ trợ người dùng hoàn chỉnh quá trình nhập liệu trên EditText
- Các bước thực hiện:
 - Khai báo dữ liệu mẫu
 - Khai báo giao diện hiển thị cho dữ liệu
 - Xây dựng Adapter tương ứng với dữ liệu và giao diện hiển thị
 - Thiết lập Adapter cho đối tượng AutoCompleteTextView



AutoCompleteTextView

▶ Ví dụ

```
// Khởi tạo dữ liệu mẫu
private static final String[] COUNTRIES = new String[]
{"VietNam", "Belgium", "France", "Italy", "Germany", "Spain"};

// Xây dựng Adapter thông qua dữ liệu mẫu và giao diện mẫu
ArrayAdapter<String> adapter = new ArrayAdapter<String> (this,
    android.R.layout.simple_dropdown_item_1line, COUNTRIES);

// Tham chiếu điều khiển
AutoCompleteTextView editCountry = (AutoCompleteTextView)
    findViewById(R.id.editCountry);

// Thiết lập Adapter cho điều khiển
editCountry.setAdapter(adapter);
```

MultiAutoCompleteTextView

- Đối tượng kế thừa từ AutoCompleteTextView
- Cho phép xây dựng dữ liệu mẫu hỗ trợ người dùng hoàn chỉnh quá trình nhập liệu trên EditText
- Dữ liệu được hỗ trợ hoàn chỉnh nhiều lần, cách nhau bằng một Tokenizer
- Các bước thực hiện:
 - Khai báo dữ liệu mẫu
 - Khai báo giao diện hiển thị cho dữ liệu
 - Xây dựng Adapter tương ứng với dữ liệu và giao diện hiển thị
 - Thiết lập Adapter cho đối tượng MultiAutoCompleteTextView
 - Thiết lập đối tượng Tokenizer

MultiAutoCompleteTextView

➤ Ví dụ:

```
// Khởi tạo dữ liệu mẫu
private static final String[] COUNTRIES = new String[]
{"VietNam", "Belgium", "France", "Italy", "Germny", "Spain"};
// Xây dựng Adapter thông qua dữ liệu mẫu và giao diện mẫu
ArrayAdapter<String> adapter = new ArrayAdapter<String> (this,
    android.R.layout.simple_dropdown_item1line, COUNTRIES);
// Tham chiếu điều khiển
MultiAutoCompleteTextView editCountry = (MultiAutoCompleteTextView)
    findViewById(R.id.editCountry);
// Thiết lập Adapter cho điều khiển
editCountry.setAdapter(adapter);
// Thiết lập Tokenizer
editCountry.setTokenizer(new MultiCompleteTextView.CommaTokenizer());
```



Thực hành



SQL

SQLite – Khái niệm

- Thẻ hiện dữ liệu ở dạng bảng quan hệ
 - Cột: trường dữ liệu (thuộc tính dữ liệu)
 - Hàng: một thẻ hiện dữ liệu

_ID	Hoten	Email	SDT
1	Nguyen Van Ti	ti@abc.com	0123456789
2	Nguyen Van Teo	teo@abc.com	0123456789
3	Nguyen Van Tun	tun@abc.com	0123456789



SQLite – Khái niệm

- Là CSDL mở được nhúng vào Android, hỗ trợ các đặc điểm về quan hệ chuẩn của CSDL như cú pháp, transaction, các câu lệnh
- Không yêu cầu về thiết lập bất cứ CSDL hoặc đòi quyền admin
- Hỗ trợ các kiểu dữ liệu: TEXT, INTEGER, REAL, BLOB.

SQLite – Khái niệm

- Đường dẫn: `DATA/data/APP_NAME/databases/FILENAME`
- CSDL SQLite cung cấp riêng cho ứng dụng tạo nó.



SQLite

- Các bước khởi tạo:
 - Tạo lớp kế thừa từ lớp SQLiteOpenHelper
 - Override phương thức onCreate() để tạo CSDL
 - Override phương thức onUpgrade() để nâng cấp CSDL
 - Lớp SQLiteOpenHelper cung cấp 2 phương thức:
 - getReadableDatabase()
 - getWritableDatabase()
 - Tạo đối tượng SQLiteDatabase để truy xuất dữ liệu
 - Cung cấp các phương thức insert(), update(), delete() hoặc execSQL() cho phép truy xuất dữ liệu



SQLite

➤ Tạo Database:

```
@Override  
public void onCreate(SQLiteDatabase _db) {  
    _db.execSQL(DATABASE_CREATE);  
}
```

➤ Mở Database:

```
SQLiteDatabase db;  
try {  
    db = dbHelper.getWritableDatabase();  
}  
catch (SQLException ex){  
    db = dbHelper.getReadableDatabase();  
}
```



Truy vấn và sắp xếp dữ liệu

Cursor – Khái niệm

- Đối tượng dữ liệu được trả về khi thực hiện truy vấn dữ liệu trong CSDL SQLite hoặc ContentProvider
- Thẻ hiện dữ liệu ở dạng bảng quan hệ
 - Cột: trường dữ liệu (thuộc tính dữ liệu)
 - Hàng: một thể hiện dữ liệu

_ID	Hoten	Email	SDT
1	Nguyen Van Ti	ti@abc.com	0123456789
2	Nguyen Van Teo	teo@abc.com	0123456789
3	Nguyen Van Tun	tun@abc.com	0123456789

Cursor – Dữ liệu Cursor từ SQLite

➤ Truy vấn CSDL SQLite:

➤ Sử dụng rawQuery():

```
Cursor cursor = getReadableDatabase().rawQuery  
("select * from todo where _id = ?", new String[] { id });
```

➤ Sử dụng query():

```
Cursor cursor = database.query  
(String table, String[] columns,  
String selection, String[] selectionArgs,  
String groupBy, String having, String orderBy);
```

Cursor – Dữ liệu Cursor từ SQLite

Truy vấn CSDL SQLite:

Sử dụng query():

```
Cursor cursor = database.query  
(String table, String[] columns,  
String selection, String[] selectionArgs,  
String groupBy, String having, String orderBy);
```

- String TABLE: tên của bảng cần truy vấn
- String[] columns: danh sách các cột sẽ trả về dữ liệu
- String selection: chứa các điều kiện truy vấn
- String[] selectionArgs: danh sách các tham số phụ cho câu điều kiện
- String[] groupBy: gom nhóm các cột kết quả
- String[] having : bộ lọc theo điều kiện
- String[] orderBy : sắp xếp theo mảng cột được chỉ định

Cursor – Dữ liệu Cursor từ SQLite

- ▶ Ví dụ: thêm dữ liệu

```
// Create a new row of values to insert.
ContentValues newValues = new ContentValues();

// Assign values for each row.
newValues.put(COLUMN_NAME, newValue);
[ ... Repeat for each column ... ]

// Insert the row into your table
myDatabase.insert(DATABASE_TABLE, null, newValues);
```

Cursor – Dữ liệu Cursor từ SQLite

- ▀ Ví dụ: cập nhật dữ liệu

```
// Define the updated row content.
ContentValues updatedValues = new ContentValues();

// Assign values for each row.
newValues.put(COLUMN_NAME, newValue);
[ ... Repeat for each column ... ]

String where = KEY_ID + "=" + rowId;

// Update the row with the specified index with the new values.
myDatabase.update(DATABASE_TABLE, newValues, where, null);
```

Cursor – Dữ liệu Cursor từ SQLite

- ▶ Ví dụ: xóa dữ liệu

```
myDatabase.delete  
(DATABASE_TABLE, KEY_ID + " = " + rowID, null);
```

Cursor – Dữ liệu Cursor từ SQLite

- ▶ Ví dụ: xóa dữ liệu

```
myDatabase.delete  
(DATABASE_TABLE, KEY_ID + " = " + rowID, null);
```

Cursor – Dữ liệu Cursor từ SQLite

➤ Một số phương thức điều khiển Cursor thao tác dữ liệu:

- moveToFirst
- moveToNext
- moveToPrevious
- moveToPosition
- getCount
- getColumnIndexOrThrow
- getColumnName
- getColumnNames
- getPosition



CursorLoader – Khái niệm

- ▶ Lớp kế thừa AsyncTaskLoader cho phép khởi tạo các câu truy vấn bất đồng bộ, dựa trên CSDL của ContentProvider
- ▶ Thực hiện truy vấn và trả dữ liệu cho FragmentActivity và Activity ở chế độ ngầm
- ▶ Các đối tượng CursorLoader được quản lý thông qua đối tượng LoaderManager tồn tại trong mỗi FragmentActivity và Activity.
- ▶ Dữ liệu trả về là một đối tượng Cursor

CursorLoader – Khái niệm

- Bao gồm 2 lớp riêng biệt được định nghĩa hỗ trợ cho các phiên bản cũ hơn của hệ điều hành

[android.content.CursorLoader](#)

- Khai báo cho các phiên bản API 11 trở lên

[android.support.v4.content.CursorLoader](#)

- Khai báo hỗ trợ cho các phiên bản API 4 trở lên

CursorLoader – Xây dựng truy vấn

- Khởi tạo: được khởi tạo thông qua LoaderManager

```
initLoader(int id, Bundle args, LoaderCallBack<D> callBacks)
```

- Trong đó:

- Id: mỗi CursorLoader khởi tạo kèm id để quản lý
- Arg: tham số hỗ trợ cho câu truy vấn
- Callback: cho phép khởi tạo câu truy vấn và nhận kết quả trả về

- Ví dụ:

```
getLoaderManager.initLoader(0, null, this);
```

CursorLoader – Xây dựng truy vấn

- Khởi tạo: được khởi tạo thông qua LoaderManager

```
initLoader(int id, Bundle args, LoaderCallBack<D> callBacks)
```

- Trong đó:

- Id: mỗi CursorLoader khởi tạo kèm id để quản lý
- Arg: tham số hỗ trợ cho câu truy vấn
- Callback: cho phép khởi tạo câu truy vấn và nhận kết quả trả về

- Ví dụ:

```
getLoaderManager.initLoader(0, null, this);
```

CursorLoader – Xây dựng truy vấn

- `LoaderManager.LoaderCallback<D>`: lớp trừu tượng cho phép thực hiện khai báo các hàm khởi tạo câu truy vấn theo kiểu dữ liệu D

Ví dụ: khai báo lớp khởi tạo truy vấn với kiểu dữ liệu `Cursor`.

```
public class MyFragmentActivity extends FragmentActivity implements
    LoaderManager.LoaderCallbacks<Cursor>
{
    ...
}
```

- Cần thực hiện override các phương thức hỗ trợ xây dựng câu truy vấn: `onCreateLoader`, `onLoadFinish`, `onLoadReset`

CursorLoader – Xây dựng truy vấn

Ví dụ khởi tạo câu truy vấn.

```
public Loader<Cursor> onCreateLoader(int id, Bundle args) {  
  
    Uri baseUri = Contacts.CONTENT_URI;  
  
    String select = "(" + Contacts.DISPLAY_NAME + " NOTNULL) AND ("  
        + Contacts.HAS_PHONE_NUMBER + "=1) AND ("  
        + Contacts.DISPLAY_NAME + " != " ");"  
  
    return new CursorLoader(getActivity(), baseUri,  
        CONTACTS_SUMMARY_PROJECTION, select, null,  
        Contacts.DISPLAY_NAME + " COLLATE LOCALIZED ASC");  
}
```

CursorLoader – Xây dựng truy vấn

- Thực hiện cập nhật dữ liệu khi có sự thay đổi.

```
public void onLoadFinished(Loader<Cursor> loader, Cursor data) {  
    mAdapter.swapCursor(data);  
}
```

- Thực hiện xoá tham chiếu dữ liệu cũ.

```
public void onLoaderReset(Loader<Cursor> loader, Cursor data) {  
    mAdapter.swapCursor(null);  
}
```



Thực hành



Broadcast Receiver & Service

Broadcast Receiver – khái niệm

- Được sử dụng để những intent message được gửi từ `sendBroadcast()` hoặc từ hệ thống
- Để đăng ký đối tượng `BroadcastReceiver`, sử dụng 2 cách:
 - Đăng ký động với phương thức: **`Context.registerReceiver()`**
 - Đăng ký trong `Manifest.xml` thông qua thẻ: **`<receiver/>`**

```
<receiver android:enabled=["true" | "false"]
          android:exported=["true" | "false"]
          android:icon="drawable resource"
          android:label="string resource"
          android:name="string"
          android:permission="string"
          android:process="string" >
    .
    .
    .
</receiver>
```



Broadcast Receiver – khai báo thuộc tính

- Có 2 thành phần chính của Broadcast có thể nhận intent:
 - Normal broadcasts: gửi theo cơ chế không đồng bộ, không theo thứ tự, sử dụng `sendBroadcast()`
 - Ordered broadcasts: mỗi thời điểm chỉ gửi đến một broadcast và sử dụng `sendOrderedBroadcast()`



Broadcast Receiver – khai báo thuộc tính

- ▶ Vòng đời: chỉ hoạt động trong thời gian `onReceiver()` được gọi, và khi kết thúc hàm này, broadcast đã hoàn thành và không còn hoạt động nữa
- ▶ Sự cung cấp quyền: chỉ broadcast receiver được cung cấp quyền mới có thể nhận những intent gửi đi

Broadcast Receiver – đăng ký và sử dụng

► Tạo Broadcast Receiver

```
public class EarthquakeReceiver extends BroadcastReceiver {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        //To do thing  
    }  
}
```

Broadcast Receiver – đăng ký và sử dụng

- Khai báo và đăng ký Broadcast Receiver

```
IntentFilter filter = new  
IntentFilter(EarthquakeService.NEW _ EARTHQUAKE_ FOUND) ;  
receiver = new EarthquakeReceiver();  
registerReceiver(receiver, filter);
```

- Hủy đăng ký cho Broadcast Receiver

```
unregisterReceiver(receiver);
```



Service – Khái niệm

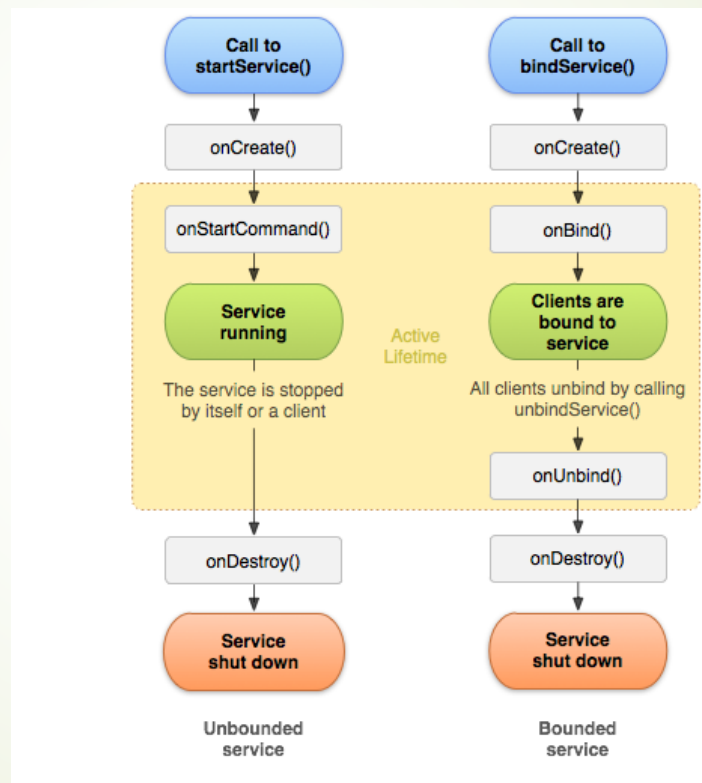
- Là một trong những thành phần của Android, được chạy trên background thực hiện những công việc tính toán không đòi hỏi giao diện
- Giống như Activity, thực hiện theo chu kỳ thời gian để quản lý sự thay đổi của trạng thái



Service – Khái niệm

- Có 2 loại service:
 - “Started” Service: service được gọi bằng `startService()` sẽ chạy vô thời hạn. Thường dùng để thực hiện công việc đơn lẻ như download, tính toán và sẽ dừng khi công việc hoàn thành
 - “bound” Service: service được gọi bằng `bindService()`. Loại service đề nghị giao diện client-server cho phép thành phần ứng dụng tương tác với nó, gửi yêu cầu và lấy kết quả. Khi tất cả các thành phần ứng dụng ngừng liên kết, service kết thúc.

Service – Vòng đời





Service – Khởi tạo

- Có thể khởi tạo service ở 2 dạng:
 - Service: lớp cơ sở của tất cả Service
 - IntentService: lớp con của Service, là cách tốt nhất nếu không đòi hỏi Service handle nhiều yêu cầu đồng thời

Service – Khởi tạo

➤ Ví dụ khởi tạo IntentService:

```
public class HelloIntentService extends IntentService{  
    public HelloIntentService() {  
        super("HelloIntentService");  
    }  
    @Override  
    protected void onHandleIntent(Intent intent) {  
    }  
}
```



Kết nối HTTP

Kết nối HTTP

- ▶ Cho phép ứng dụng kết nối và trao đổi dữ liệu đến một máy chủ thông qua giao thức http
- ▶ Máy chủ đóng vai trò cung cấp Web service
- ▶ Thường sử dụng HTTP request: GET, POST để gửi và nhận dữ liệu từ máy chủ.
- ▶ Cần cung cấp quyền trong file Manifest.xml

```
<uses-permission android:name="android.permission.INTERNET"/>
```

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
```



Thực hành